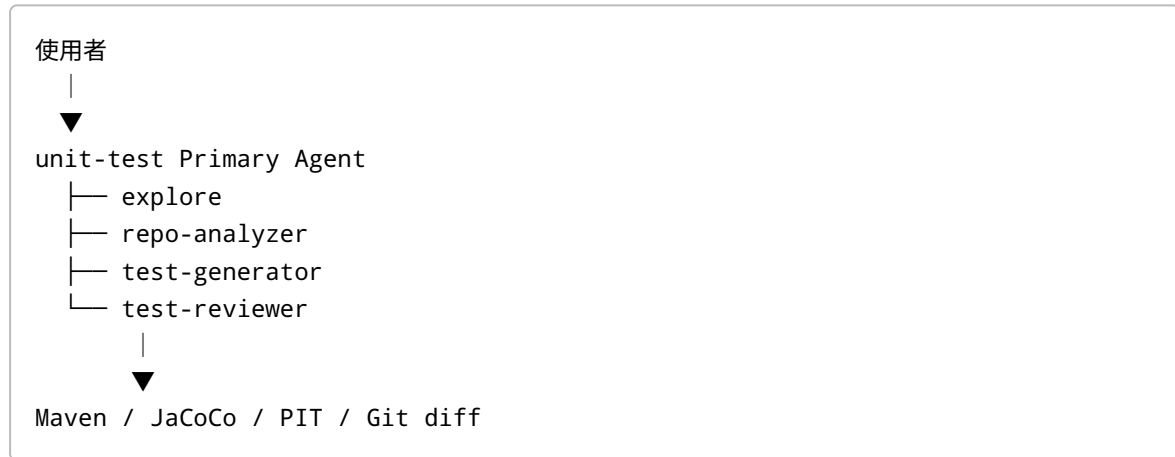


# Legacy Repo Unit Test Agent Plan

## 架構



## Agent 職責

### unit-test Primary Agent

- 接收測試任務。
- 委派 Subagent。
- 分類規格與現有行為。
- 要求工程師確認規格衝突。
- 執行測試與品質驗證。
- 只能修改 `src/test/**`。

### repo-analyzer Subagent

- 唯讀分析 Repo。
- 找出分支、邊界、依賴與既有測試。
- 每項規則附上來源。
- 不得把現有 Code 直接視為正確規格。

### test-generator Subagent

- 根據已核准的測試意圖產生 JUnit 5 / Mockito 測試。
- 只能修改 `src/test/**`。
- 不得自行修改預期結果。
- 不得修改 production code。

### test-reviewer Subagent

- 唯讀審查測試品質。
- 檢查弱 Assertion、過度 Mock、缺少邊界案例與 Mutation Survivor。
- 檢查 Git diff 是否包含未授權修改。

## 流程

選定目標

- 分析 Repo
- 分類規格與行為
- 人工確認
- 產生測試
- Maven 編譯與測試
- JaCoCo
- PIT
- 品質審查

規則分為：

| 分類                      | 處理方式                       |
|-------------------------|----------------------------|
| Confirmed Specification | 產生正式 Unit Test             |
| Current Behavior        | 只能建立 Characterization Test |
| Suspected Behavior      | 等待人工確認                     |
| Conflict                | 阻擋測試產生                     |

## OpenCode 目錄

```
repo-root/
├── AGENTS.md
├── PLAN.md
├── opencode.json
├── .opencode/
│   ├── agents/
│   │   ├── unit-test.md
│   │   ├── repo-analyzer.md
│   │   ├── test-generator.md
│   │   └── test-reviewer.md
│   ├── commands/
│   │   ├── unit-test-analyze.md
│   │   ├── unit-test-generate.md
│   │   └── unit-test-review.md
│   └── skills/
│       ├── java-unit-testing/
│       └── SKILL.md
```

## 導入順序

### Phase 1：唯讀分析

建立 `repo-analyzer` 與 `/unit-test-analyze`，不得產生任何 Git diff。

### Phase 2：受限測試產生

建立 `test-generator`，只開放 `src/test/**`，並執行 Maven 編譯與指定測試。

### Phase 3：品質驗證

加入 JaCoCo、PIT、`test-reviewer` 與 Git diff 稽核。

### Phase 4：外部 Orchestrator

出現嚴格流程、持久化、CI 批次、多 Repo 或重試控制需求後，再導入 OpenCode SDK、LangGraph 或自製 Orchestrator。

## MVP 決策

第一版採用：

- 1 個 Primary Agent
- + 3 個自訂 Subagent
- + 內建 explore
- + 人工規格核准
- + Maven / JaCoCo / PIT

第一版不自動修改正式程式、不修改 `pom.xml`、不建立 PR、不掃描並改寫整個 Repo。